

RESUMO PARA PROVA

ACH2005 - Analise, Projeto e Interface Humano-Computador

Aula 08 - Aprendizagem, Resolucao de Problemas, Leitura, Fala e Escrita

VISAO GERAL

Topico	Conceito central
Aprendizagem	Acumulo de habilidades e conhecimentos; aprender fazendo e mais eficaz do que ler manuais
Resolucao de problemas	Envolve cognicao reflexiva; pessoas usam heurísticas simples, nao avaliacao racional completa
Leitura, fala e escuta	Tres formas de processar linguagem com diferentes custos cognitivos e implicacoes para design

1. APRENDIZAGEM

A aprendizagem consiste no acumulo de habilidades e conhecimentos. Pode ser intencional ou nao (psicologia cognitiva), mas no desenvolvimento de software **nao se deve assumir que o usuario simplesmente ira aprender** — isso requer esforco consciente. Fato central: **pessoas aprendem mais fazendo do que lendo manuais**.

1.1 Principios da aprendizagem:

Principio	Descricao	Implicacao para design
Aprender pela experiencia e (geralmente) facil	Pessoas sao boas em generalizar e extrair conclusoes de experiencias especificas. Limitacoes: situacoes complexas, influencia de experiencias alheias, erros que nao ensinam, generalizacoes com dados incompletos	Design deve reduzir complexidade e guiar o usuario para que ele aprenda pela experiencia de uso
Executar acoes ja aprendidas e facil	Acoes automatizadas exigem custo cognitivo quase zero e podem ser feitas em paralelo. Praticando, qualquer acao pode se tornar automatica	Aproveite padroes ja conhecidos pelo usuario; nao force-o a aprender novos modos de fazer tarefas familiares
Executar novas acoes e dificil	Pessoas aprendendo novas habilidades sentem-se sobrecarregadas. Quando ha pressa, preferem metodos ja automaticos mesmo que haja alternativas melhores	Reduza o numero de novidades por vez; use revelacao progressiva; nao force mudancas abruptas em interfaces ja conhecidas
Pratica frequente, regular e precisa	Frequente: uso raro dificulta aprendizado; frequente = familiaridade rapida. Regular: 3-40 semanas para automatizar, dependendo da complexidade. Precisa: pratica imprecisa prejudica o aprendizado	Projete para uso diario se quiser familiaridade rapida; guie caixas eletronicas e sites de uso raro; ferramentas que ajudem o usuario a ser preciso

Vocabulário adequado, familiar e consistente	"Mesmo nome, mesma coisa; nome diferente, coisa diferente". Terminologia deve corresponder as expectativas do usuário e ser consistente em todo o sistema	Não crie termos novos desnecessários; mantenha nomenclatura uniforme em todo o produto
Risco baixo = mais exploração	Usuários exploram mais quando não tem medo de cometer erros. A maioria dos sistemas tem mais funcionalidades do que os usuários usam, pois o medo de errar inibe a exploração	Crie barreiras a erros; desative comandos inválidos; mostre consequências claramente; permita desfazer (undo) ações
Revelação progressiva e metáforas	Iniciar com poucos controles e aumentar a complexidade conforme o usuário ganha experiência. Metáforas conectam conceitos novos a estruturas já conhecidas	Onboarding simplificado; funções avançadas escondidas até o usuário estar pronto; use metáforas do mundo físico (ex: lixeira, pasta, envelope)

1.2 Implicações para o design (Aprendizagem):

#	Diretriz	Como aplicar
1	Indique estado do sistema e progresso do usuário	Mostre claramente em que etapa o usuário está e o quanto falta para atingir seu objetivo
2	Guie o usuário em direção aos objetivos	Use "wizards" e fluxos guiados; não apresente múltiplas opções de igual importância esperando que o usuário saiba por onde começar
3	Diga o que o usuário precisa saber de forma clara	Não espere que o usuário deduza ou adivinhe informações por processo de eliminação
4	Não force o usuário a diagnosticar problemas do sistema	Usuários geralmente não têm conhecimento técnico para isso; o sistema deve diagnosticar e comunicar o problema
5	Diminua número e complexidade de configurações	Não espere que usuários combinem configurações para otimizar o uso; defaults bem pensados são fundamentais
6	Projete para atingir familiaridade	Use conceitos, terminologias, imagens e gráficos já conhecidos; siga padrões de plataforma; use metáforas
7	Projete para atingir consistência	Elementos, telas e processos consistentes = aprendizado mais rápido e menor carga cognitiva
8	Promova a exploração da interface	Barreiras a erros, undo fácil, consequências claras = usuário confiante para explorar funcionalidades

2. RESOLUÇÃO DE PROBLEMAS, PLANEJAMENTO E TOMADA DE DECISÃO

Resolução de problemas, planejamento, raciocínio e tomada de decisão envolvem **cognição reflexiva** (esforço mental, atenção, julgamento). Inclui: pensar no que fazer, avaliar opções, definir procedimentos, prever consequências e monitorar o progresso.

Teorias racionais vs. heurísticas:

Abordagem	Como funciona	Implicação para design
Teoria racional	Tomada de decisão = avaliar e comparar todas as opções disponíveis; processo muito custoso cognitivamente	Não projete interfaces que exijam avaliação exaustiva de todas as opções

Heurísticas (realidade)	Pessoas usam atalhos mentais simples para tomar decisões "boas o suficiente". Ex: compram marcas conhecidas, com preço razoável e embalagem atraente, sem ler todas as informações	Destaque a informação-chave; o que e destaque pode variar por usuário (ex: alérgicos procuram ingredientes específicos)
-------------------------	--	---

Sobre cálculos: problemas de 1-2 passos são fáceis para a maioria das pessoas. Cálculos com mais de 2-3 passos com uso intenso de memória de trabalho são difíceis — as pessoas recorrem a papel, diagramas ou ferramentas externas.

Implicações para o design (Resolução de Problemas):

#	Diretriz	Como aplicar
1	Deixe o computador fazer os cálculos	Não force o usuário a calcular o que o sistema pode calcular automaticamente (ex: totais, descontos, datas)
2	Use percepção, não cálculo	Problemas que parecem precisar de cálculo podem ser representados graficamente (ex: barras de progresso, gráficos comparativos) — percepção visual é muito mais rápida
3	Ofereça ajuda fácil de acessar	Páginas de ajuda e documentação para usuários que querem entender como executar tarefas de forma eficiente
4	Funções simples e fáceis de lembrar	Apoie a tomada de decisão rápida e o planejamento com funções diretas e sem ambiguidade
5	Permita salvar critérios de decisão	Deixe usuários definir e salvar preferências ou filtros para decisões recorrentes (ex: filtros de busca salvos)

3. LEITURA, FALA E ESCUTA

São três formas de processar a linguagem. Similaridade: o significado de uma sentença é o mesmo independentemente da forma em que é processada. A facilidade de cada forma varia com a pessoa, a tarefa e o contexto.

Forma	Características-chave	Custo cognitivo
Leitura	Linguagem escrita e permanente (pode ser revisitada); tende a ser gramatical; pessoas com dislexia têm dificuldade em reconhecer palavras escritas	Moderado; pode ser mais fácil que escrita ou escuta em alguns contextos
Fala	Geralmente não-gramatical; mais natural e rápida de produzir; interfaces de voz exigem mais memória de trabalho	Alto para produção; moderado para recepção
Escuta	Transiente (desaparece após ser emitida, não pode ser "relida"); requer menos esforço cognitivo do que ler ou falar	Baixo; porém a informação não persiste — dificulta revisão

Implicações para o design (Leitura, Fala e Escuta):

- Crie menus e instruções com poucas opções** — limitar a quantidade de itens reduz a carga cognitiva de leitura e escuta.
- Áudios gerados automaticamente devem ter boa entonação** — entonação adequada facilita a compreensão da linguagem falada.
- Permita configuração de tamanho de textos e volume** — acessibilidade para diferentes necessidades visuais e auditivas.

10 PONTOS CRITICOS PARA PROVA DE MULTIPLA ESCOLHA

1. Aprender fazendo > aprender lendo

Pessoas aprendem mais pela experiencia direta do que por manuais. O design deve facilitar o aprendizado pela exploracao do proprio sistema.

2. Acoes automaticas = custo cognitivo quase zero

Acoes ja aprendidas sao executadas quase sem esforco e podem ser feitas em paralelo. Acoes novas sobrecarregam o usuario — por isso ele prefere o caminho familiar mesmo quando ha um melhor.

3. Risco baixo = mais exploracao

Usuarios exploram mais quando nao temem errar. Design deve: criar barreiras a erros, desativar comandos invalidos, mostrar consequencias e permitir undo.

4. Prática: 3 a 40 semanas para automatizar

O tempo para uma acao se tornar habito automatico depende da complexidade. Pratica diaria acelera o processo. Sistemas de uso raro devem guiar mais o usuario.

5. Vocabulario: mesmo nome, mesma coisa

Terminologia inconsistente sobrecarrega a aprendizagem. Use vocabulario familiar ao usuario, nao jargao tecnico, e seja consistente em todo o produto.

6. Revelacao progressiva

Comece com poucos controles e va revelando complexidade conforme o usuario ganha experiencia. Evita sobrecarga no aprendizado inicial.

7. Heurísticas, nao logica racional

Pessoas nao avaliam todas as opcoes — usam atalhos mentais (heurísticas). Design efetivo: destaque a informacao-chave, pois o usuario decide rapido baseado no que ve primeiro.

8. Calculos de mais de 2-3 passos sao dificeis

Use o computador para calcular; use graficos para substituir calculos — percepcao visual e muito mais rapida e menos custosa que calculo mental.

9. Escuta: transiente; leitura: permanente

Informacao ouvida desaparece e nao pode ser revisitada. Escuta requer menos esforco cognitivo, mas nao e adequada para informacoes que precisam ser revisadas.

10. Nao force o usuario a diagnosticar problemas do sistema

Usuarios nao tem conhecimento tecnico para isso. O sistema deve identificar, comunicar e sugerir solucao para seus proprios problemas de forma clara.